

```
def walk_rows_for_connect4(board):
    return walk_slices_for_
connect4(board, len(board),
len(board[0]))

def walk_cols_for_connect4(board):
    return walk_slices_for_
connect4(board, len(board[0]),
len(board),
is_
reversed=True)
```

While the above code looks reasonably modular, consider the complexity of extending this for the diagonals. By employing the *is_reversed* flag to swap the order of indices, we have managed to handle rows and columns. However, the value of indices for diagonals is quite different and that requires us to extend the definition of the simple flag to cover all the three types of slices. Such a solution is not very desirable, as it moves the complexity of slice construction to the slice walker.

One way this can be circumvented is to build a fresh slice from the original board cells. Given the limited board size, that approach may be acceptable in this example. But for large matrices, creating temporary slices can be quite costly in terms of both CPU and memory usage. Hence, it is not workable, in general. In the next section, we see how Python's generators can help address this situation.

Solution using Python generators

We now rewrite the solution to our problem in terms of generators. These are used to generate an arbitrary slice that the *check* function operates on, which provides a clean separation of concerns. The final implementation using generators is shown below.

```
def col_generator(board, col):
    rows = len(board)
    for r in range(rows):
        yield (board[r][col], (r,
```

```
col),),)

def row_generator(board, row):
    cols = len(board[0])
    for c in range(cols):
        yield (board[row][c], (row,
c),),)

def check_4connected(board, loc, slice_
generator):
    c, count = None, 0
    for t in slice_generator(board,
loc):
        c, count = compare_
element(t[0], c, count)
        if count == 4 and c != 'b':
            return c, t[1][0], t[1][1]
    return (None, -1, -1)

def walk_slices_for_connect4(board,
n_slices, generator_fn):
    for i in range(n_slices):
        c, x, y =

check_4connected(board, i, generator_
fn)

        if c:
            return (c, x, y)

    return (None, -1, -1)

def walk_rows_for_connect4(board):
    return walk_slices_for_
connect4(board, len(board), row_
generator)

def walk_cols_for_connect4(board):
    return walk_slices_for_
connect4(board, len(board[0]), col_
generator)
```

If we consider the board to contain different top-level features like rows, columns and diagonals, then the above implementation can be mapped as:

- The top level functions like *walk_rows_for_connect4* and *walk_cols_for_connect4* express the intent to search for 4-connectedness over board features like, rows, columns and diagonals.
- The second level function *walk_*

slices_for_connect4 realises the above by walking multiple instances of the same feature, regardless of the type of the feature.

- The third level function *check_4connected* performs the actual checking in the context of one instance of a feature.
- Generator functions, like *row_generator* or *col_generator*, which are the third level functions, call out into the generator to dynamically iterate on the feature-specific slice. Both the second and third level functions are intentionally kept agnostic of how the slice is obtained.

With this design, it is easier to implement walking both kinds of diagonals. For each diagonal type, a top-level function and a generator function have to be implemented. The second and third level functions just remain unchanged and work seamlessly. Readers are encouraged to try out the generator functions for the diagonals and then compare their work with a solution posted by the author (see References).

Using a real-life example, we have seen how Python's generator functions can be used to realise and reap the benefits of key software design principles like the 'separation of concerns'. As mentioned at the outset, leveraging this Python feature led to an elegant implementation that promotes extensibility and maintainability of the code. **END** 🐧

References

- [1] Python generator wiki: <https://wiki.python.org/moin/Generators>
- [2] Complete solution for connect4: <https://github.com/srgorti/connect4>

By: Sriram Gorti

The author is an avid fan of the simplicity and ease of programming with Python and tinkers with it on weekends. On weekdays, he maintains and extends production code bases for email and Web gateways, significant portions of which are in Python.